

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Sentiment Analysis (NLP)

Workshop No. 1 — Data Preparation & Supervised Learning Baselines

---

Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Course: Machine Learning

Professor: Carlos Andres Sierra Virguez

October, 2026

Xiomara Salomé Arias Arias	20222020028
Julian David Celis Giraldo	20222020041
Alejandro Gómez Moyano	20222020073

Contents

1 Executive Summary **2**

2 Dataset Description **2**

3 Exploratory Data Analysis **2**

4 Preprocessing Pipeline **6**

5 Baseline Model Results **10**

6 Discussion **11**

1 Executive Summary

To evaluate baseline performance on an imbalanced, left-skewed dataset of Disneyland park reviews, rating values were transformed into a three-class sentiment scheme (positive, neutral, and negative). Four baseline models were selected: a Dummy Classifier (majority strategy) to set the minimum performance floor, a Logistic Regression model using a softmax/sigmoid function for class probabilities, a Random Forest Classifier combining ensemble decision trees, and a Multinomial Naive Bayes classifier predicting labels based on word frequencies.

Logistic Regression resists classifying everything as the most frequent sentiment. Random Forest collapses into the majority class due to the imbalance, which reweighting inside bootstrap samples could fix. Multinomial Naive Bayes: Has the best accuracy, but it is not discriminative due to its mode of work as a probabilistic model, so this tends to be its weakness.

The models evaluated are insufficient for the job of classifying sentiment in the reviews, with macro-F1 as the metric to judge performance, macro-f1 gives equal importance to each class, showing the difficulty of classifying a neutral review.

The repository for this workshop is available at: https://github.com/JulianCelisCreator/ML-NLP-Sentimental-Analysis.git

2 Dataset Description

Description: Reviews and Ratings of 3(Range 1-5, 5 is the high score) Disneyland branches - California, Hong Kong and Paris

Table 1: Dataset Provenance and Metadata

Property	**Details**
Source	Kaggle (`arushchillar/disneyland-reviews`)
License	CC0: Public Domain
URL	https://www.kaggle.com/datasets/arushchillar/disneyland-reviews
Shape	42,656 reviews × 6 columns

definition of classes:

Table 2: Target Sentiment Distribution

Rating	**count**	**pct (%)**	**ratio_vs_smallest**
positive	33,921	79.52	9.4
neutral	5,109	11.98	1.4
negative	3,626	8.50	1.0

Features information:

3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to examine the distribution of the target variable, the characteristics of the review text, and potential data quality and leakage issues. The dataset

Table 3: Dataset Columns and Value Ranges Summary

Column	Dtype	Role / Value Range
Review_ID	int64	identifier (per review, NOT per user) [1 – 42,636]
Rating	int64	ordinal 1–5 score; source of sentiment target (1–2: negative, 3: neutral, 4–5: positive)
Year_Month	str	visit period as 'YYYY-M'; temporal range: 2010 to 2019 (112 unique months)
Reviewer_Location	str	reviewer's self-declared country (162 unique locations)
Review_Text	str	free-text review body; the only model input
Branch	str	Disneyland park location (3 branches: Disneyland_HongKong, Disneyland_California, Disneyland_Paris)

contains 42,656 reviews across six variables: `Review_ID`, `Rating`, `Year_Month`, `Reviewer_Location`, `Review_Text`, and `Branch`.

Target Distribution

The rating distribution is strongly imbalanced, with 54.3% of the reviews receiving five stars and 25.3% receiving four stars.

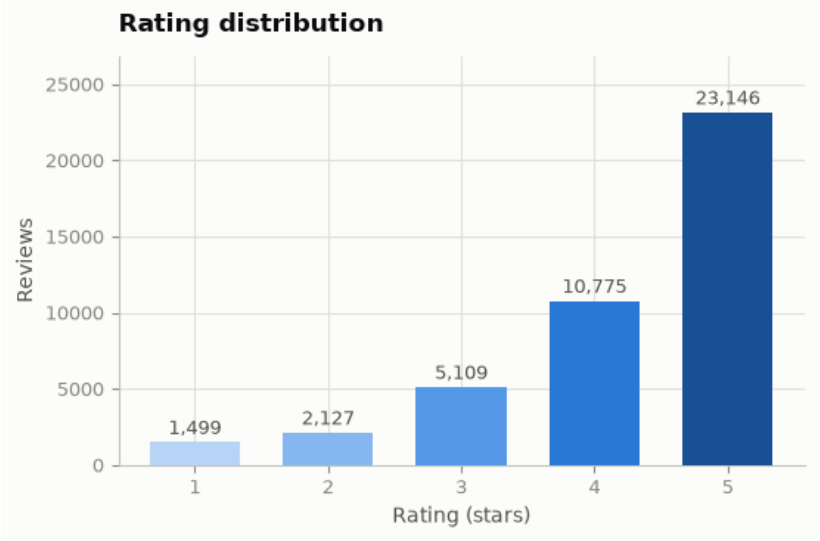


Figure 1: Distribution of review ratings.

After mapping the ratings into three sentiment classes, the imbalance becomes even more evident: 79.5% of the reviews are positive, 12.0% are neutral, and 8.5% are negative.

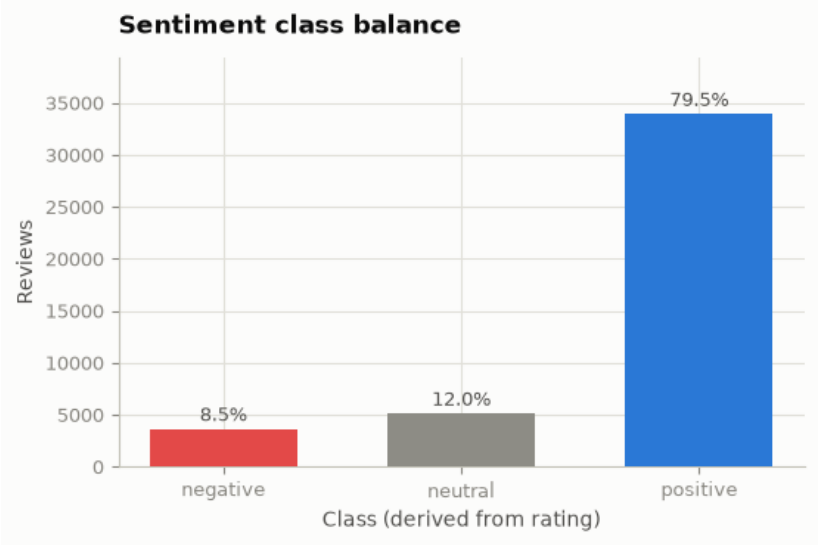


Figure 2: Distribution of sentiment classes.

This distribution means that a classifier predicting only the majority class would achieve 79.5% accuracy without using the review text. Therefore, accuracy is not sufficient as the primary evaluation metric, and macro-F1 is used to give equal importance to all sentiment classes.

Text Characteristics

The reviews contain approximately 130 words and 697 characters on average, with approximately seven sentences per review. However, the distribution is highly dispersed, with reviews reaching up to 3,963 words.

The relationship between review length and rating was also examined. One-star reviews have a median length of 147 words, compared with 67 words for five-star reviews. The Spearman correlation between review length and rating is $\rho = -0.23$, indicating an association between the two variables.

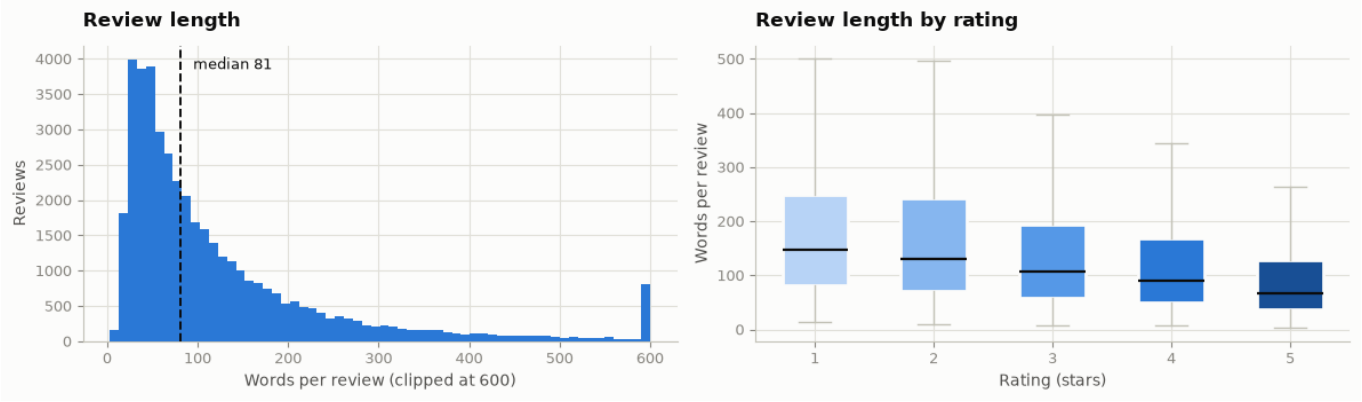


Figure 3: Review length distribution by rating.

Relationships and Leakage Indicators

The EDA identified several relationships that are relevant to feature selection and model validity.

First, the distribution of sentiment differs substantially between Disneyland branches. Negative reviews represent 13.7% of reviews from Disneyland Paris compared with 6.4% from California.

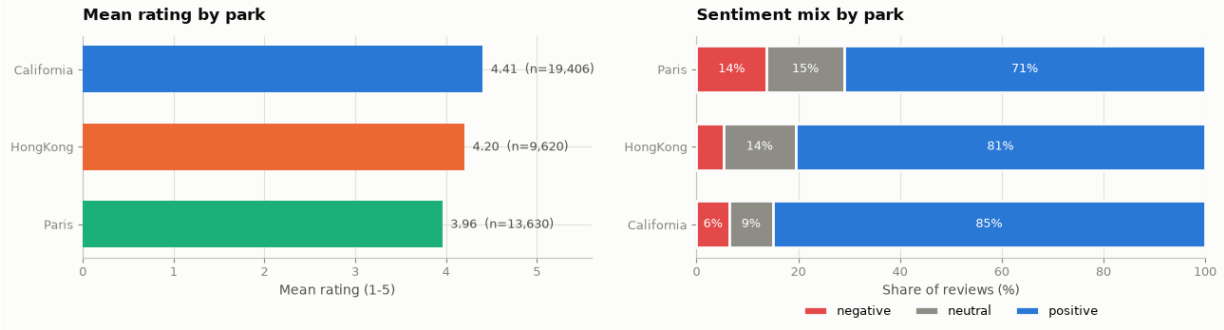


Figure 4: Sentiment distribution by Disneyland branch.

This indicates that `Branch` may act as a confounding variable, allowing a model to infer sentiment from the location rather than from the review text itself.

Second, `Review_ID` presents a Spearman correlation of $\rho = +0.9927$ with the review month. This indicates that the identifier contains strong temporal information and should therefore be excluded from the predictive features.

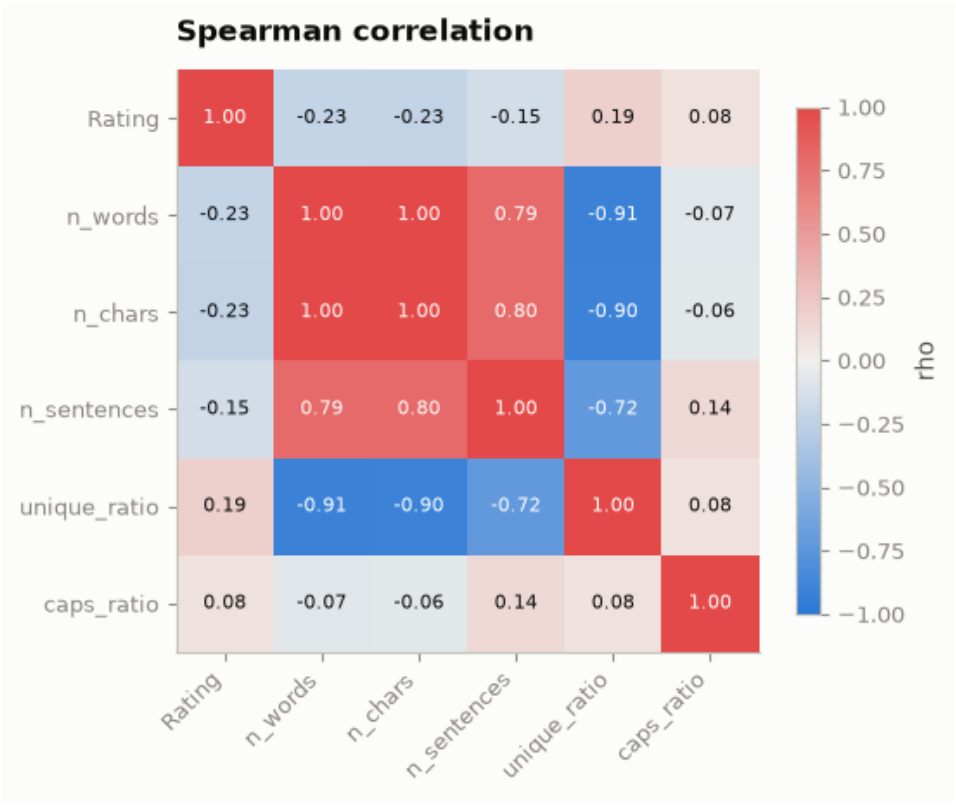


Figure 5: Correlation matrix of numerical variables.

The main EDA findings relevant to the modeling process are summarized in Table 4.

Table 4: Main EDA findings and their implications.

Finding	**Observation**	**Implication**
Class imbalance	Positive reviews represent 79.5% of the dataset	Use macro-F1 as the main evaluation metric.
Review length	Negative reviews tend to be longer; $\rho = -0.23$	Retain length information, but handle extreme values.
Branch distribution	Paris has 13.7% negative reviews vs. 6.4% in California	Exclude `Branch` as a predictive feature.
Review ID	$\rho = +0.9927$ with review month	Exclude `Review_ID` to prevent leakage.
Feature redundancy	`n_words` and `n_chars`: $\rho = 1.00$	Remove one redundant feature.
Duplicate reviews	24 duplicated review texts	Remove before train-test splitting.

Data Quality

Although the dataset initially reports zero null values, the EDA revealed several non-null quality issues. The most relevant issue is the presence of the literal value "missing" in `Year_Month`, affecting 2,613 records (6.13%). These values were not detected by standard null-value checks and therefore required explicit conversion to missing temporal values.

Table 5: Data quality issues identified during EDA.

Issue	**Rows**	**Percentage**
"missing" in `Year_Month`	2,613	6.13%
Reviews longer than 1,000 words	172	0.40%
Reviews containing URLs	59	0.14%
Duplicated review texts	24	0.06%
Duplicated `Review_ID` values	20	0.05%
Completely duplicated rows	12	0.03%

The duplicated review texts were considered a potential source of data leakage and are removed before splitting the dataset. Reviews identified as length outliers using the IQR criterion were not removed, since long reviews can represent valid user feedback. Instead, they are handled through truncation during the modeling stage.

No null or empty review texts, invalid ratings, or emojis were detected. Overall, the dataset is relatively clean, with the main preprocessing considerations being class imbalance, duplicated content, temporal information embedded in `Review_ID`, and the non-null "missing" values in `Year_Month`.

4 Preprocessing Pipeline

The preprocessing pipeline was designed with two separate phases to prevent data leakage. The first phase applies transformations that represent intrinsic properties of the dataset, such as duplicate

removal, missing-value handling, and target construction. The second phase contains parameters that must be learned exclusively from the training set, such as the TF-IDF vocabulary, feature scaling parameters, and class weights. These parameters are subsequently applied to the validation and test sets without refitting them.

This separation ensures that no information derived from the validation or test sets is used during model training.

Data Cleaning

The first stage removes redundant observations and addresses non-standard missing values.

Table 6: Data cleaning decisions.

Decision	**Action**	**Justification**
Exact duplicates	12 rows removed	They provide no additional information.
Duplicated review texts	24 rows removed before splitting	Prevents identical texts from appearing in different dataset partitions and artificially inflating performance.
"missing" sentinel	2,613 values converted to `NaT`	The dataset stores missing periods as a literal string rather than a null value.

Duplicated review texts are removed before the split because they represent a direct risk of data leakage. If the same review appears in both training and test sets, the model may effectively encounter the same information during training and evaluation.

Target Construction

A three-class sentiment target was constructed from the original rating:

Table 7: Mapping from rating to sentiment.

Rating	**Sentiment**
1–2	Negative
3	Neutral
4–5	Positive

The three-class formulation preserves the neutral category, which is linguistically meaningful and represents a more challenging classification problem than a binary formulation. The original `Rating` variable is used only to construct the target and is not included as a model feature, since doing so would directly provide the answer to the classifier.

Text Normalization

The review text was normalized before feature extraction. The main transformations were:

- **Lowercasing:** words such as "Amazing" and "amazing" are treated as the same token, reducing unnecessary vocabulary variation.
- **URL removal:** URLs were removed because they do not provide relevant sentiment information for this task.
- **Negation preservation:** negation words such as "not" and "no" were retained because removing them can change the meaning of expressions such as "not good".

Feature Engineering

The model input combines textual features with a small set of numerical features describing writing style.

Table 8: Feature engineering decisions.

Feature / Decision	**Justification**
TF-IDF	Converts text into numerical features while assigning higher weights to informative terms and lower weights to very common terms.
Number of words	Captures differences in review length and writing style.
Exclamation marks	May capture emotional intensity in the review.
Uppercase characters	May indicate emphasis or emotional expression.
Remove `n_chars`	Perfectly correlated with `n_words` ($\rho = 1.00$), making it redundant.
Exclude `Branch`	Its distribution is associated with sentiment and could allow the model to infer sentiment from the park rather than the text.
Exclude `Review_ID`	Strongly correlated with review time ($\rho = 0.9927$), making it a potential leakage source.
Exclude `Reviewer_Location`	Avoids introducing contextual information unrelated to the textual sentiment task.

Train-Only Learned Parameters

To prevent data leakage, parameters that are estimated from the data are learned exclusively from the training set.

Table 9: Parameters learned exclusively from the training set.

Parameter	**Strategy**
TF-IDF vocabulary and weights	Fitted on the training set and then applied to validation and test data.
StandardScaler	Fitted using training data only; the same transformation is applied to validation and test data.
Class weights	Calculated from the training-set class distribution using balanced weighting.

The class weights address the 9.4:1 imbalance between the positive and negative classes without resampling the dataset. This avoids introducing duplicated or synthetic observations across dataset partitions.

Dataset Splitting

The dataset is divided into training, validation, and test sets using a stratified 70/15/15 split with `random_state=42`. Stratification preserves the sentiment distribution across all three partitions, which is particularly important because the negative class represents only 8.5% of the dataset.

The resulting partitions are stored in `splits.json`. This ensures that all experiments and models use the same data partitions rather than generating a new random split for each experiment.

The order of the preprocessing pipeline is summarized below:

1. Remove exact and text-level duplicates.
2. Convert the "missing" sentinel values in `Year_Month`.
3. Construct the three-class sentiment target.
4. Exclude variables that may introduce leakage or unwanted confounding information.
5. Perform the stratified 70/15/15 train-validation-test split.
6. Fit the TF-IDF vectorizer using training data only.
7. Fit the numerical scaler using training data only.
8. Calculate class weights using the training-set distribution.
9. Apply the learned transformations to validation and test data.

This order ensures that transformations involving learned parameters never use information from the validation or test sets.

Data Augmentation

No data augmentation was applied. Techniques such as back-translation or synonym replacement can modify the semantic meaning of a review and potentially alter its sentiment label. Additionally, the dataset contains 42,656 reviews, providing sufficient observations for establishing baseline models. The class imbalance is instead addressed through class weighting during model training.

5 Baseline Model Results

float

Three supervised baselines and a majority-class reference were trained on TF-IDF features of `Review_Text` (scikit-learn's default TF-IDF vectorizer: unigrams, 35,605 columns). Logistic Regression and Random Forest use balanced class weights. All numbers are on the validation split; the test split is kept untouched for later workshops.

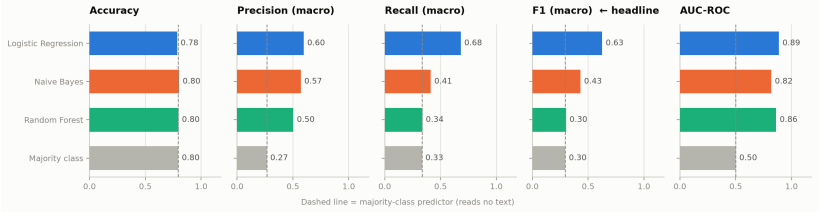


Figure 6: Baseline metrics per model.

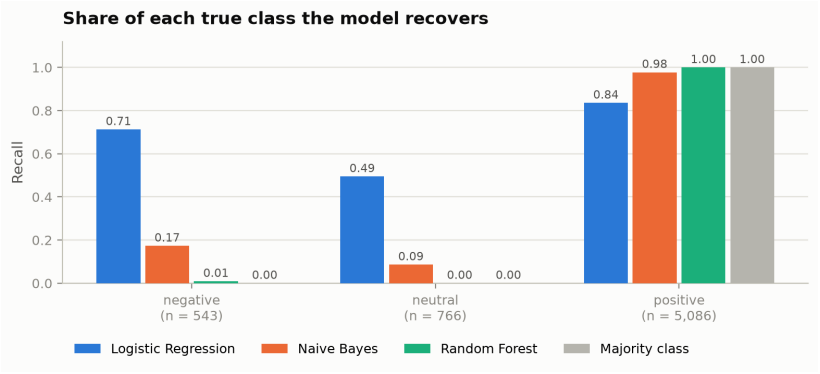


Figure 7: Baseline recall metrics per model.

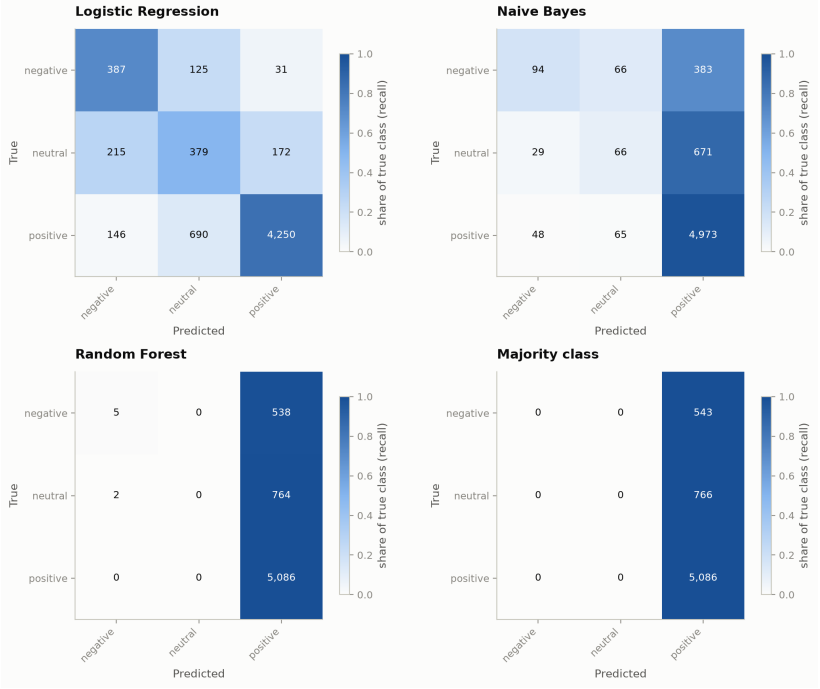


Figure 8: Baseline confusion metrics per model.

6 Discussion

Problem difficulty: accuracy is misleading here. Positive reviews make up 79.5% of the data, so a classifier that never reads a review reaches 0.7953 accuracy. Logistic Regression scores *below* that (0.7844), yet it is the only model that actually learns the task: its macro-F1 is 0.63 against the majority's 0.30. Naive Bayes has the highest accuracy of all four models (0.8027) and a macro-F1 of only 0.43. Ranking by accuracy would have picked one of the weakest classifiers. Random Forest makes the same point from another angle. Its AUC of 0.86 says its probability ranking is sound, while its macro-F1 of 0.30 says its argmax decisions are no better than the majority baseline: it labels all but 7 of 6,395 reviews as positive. The gap points at the decision threshold, not at the representation.

Class balance: neutral is the hardest class. Even the best model recalls only 0.49 of neutral reviews, and neutral precision is 0.32. The errors run in both directions. 215 neutral reviews are labelled negative and 172 positive. In the other direction, 690 positive reviews are pulled into neutral, which accounts for most of Logistic Regression's accuracy loss. This is a side effect of balanced class weights: they buy minority-class recall with positive-class precision. A 3-star review usually mixes praise and complaints ("great rides, but the queues…"), so its vocabulary overlaps with both ends of the scale. Neutral is the class a bag-of-words model is least equipped to separate.

Features: how far bag-of-words goes. TF-IDF unigrams get Logistic Regression to 0.71 recall on negative reviews, so the sentiment signal is largely lexical. But bag-of-words drops word order. It cannot represent negation ("not worth it"), contrast ("good but overpriced") or which clause carries the final verdict, and those are exactly the cues that separate a 3-star review from a 2- or 4-star one. A sequential model in Workshop 2 (LSTM or transformer) should therefore improve *neutral*

recall and F1 most of all, rather than overall accuracy. Workshop 2 will be compared against these numbers on the same split, seed and metric.

Why macro-F1 is the right metric. Macro-F1 averages the three per-class F1 scores without weighting, so the negative class (8.5% of reviews) counts as much as the positive class (79.5%). A model that ignores the minority classes cannot score above ≈ 0.30, which is the majority baseline. The metric was fixed during the EDA, before any model was trained, so it was not chosen after seeing which model it favoured. AUC-ROC is reported alongside because the brief requires it and because it separates ranking quality from decision quality, as the Random Forest case shows.